

## Unit 5

### Introduction to Convolution Neural Network (CNN)

A Convolutional Neural Network (CNN) is a type of Deep Learning neural network architecture commonly used in Computer Vision. Computer vision is a field of Artificial Intelligence that enables a computer to understand and interpret the image or visual data.

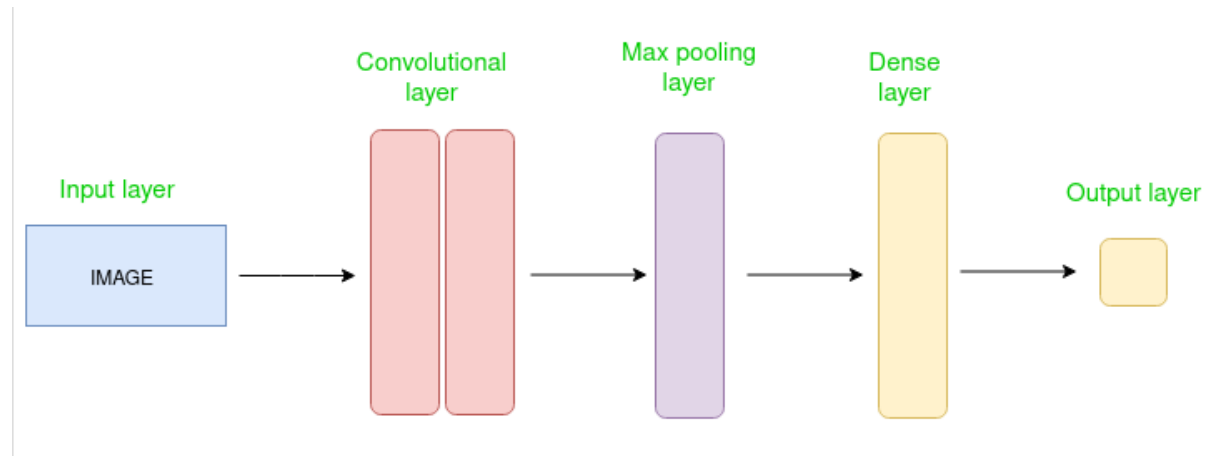
Convolutional Neural Network (CNN) is the extended version of artificial neural networks (ANN) which is predominantly used to extract the feature from the grid-like matrix dataset. For example visual datasets like images or videos where data patterns play an extensive role.

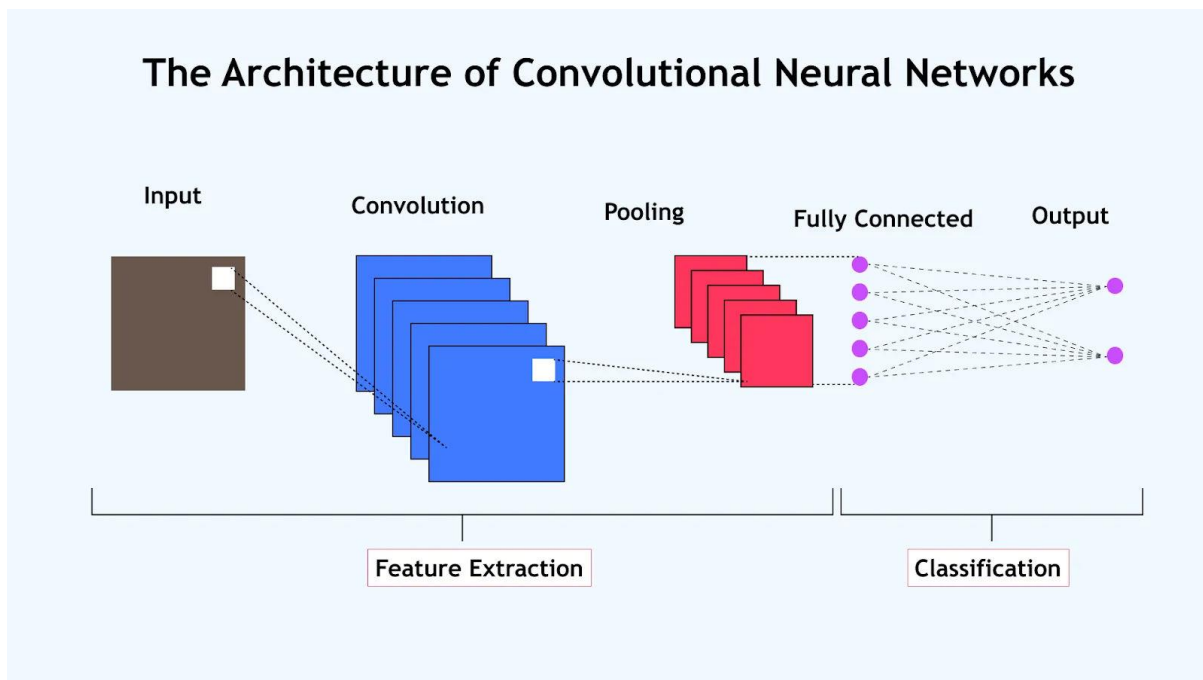
Neural Networks are used in various datasets like images, audio, and text. Different types of Neural Networks are used for different purposes, for example for predicting the sequence of words we use Recurrent Neural Networks more precisely an LSTM, similarly **for image classification we use Convolution Neural networks.**

### CNN Architecture

Convolutional Neural Network consists of multiple layers like the input layer, Convolutional layer, Pooling layer, and fully connected layers.

The Convolutional layer applies filters to the input image to extract features, the Pooling layer down samples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.





### Types of layers

Let's take an example by running a **covnets** on of image of dimension  $32 \times 32 \times 3$ .

**Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with **width 32, height 32, and depth 3**.

**Convolutional Layers:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually  $2 \times 2$ ,  $3 \times 3$ , or  $5 \times 5$  shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension  $32 \times 32 \times 12$ .

**Activation Layer:** By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the **output of the convolution layer**. Some common activation functions are RELU:  $\max(0, x)$ , Tanh, Leaky RELU, etc. The volume remains unchanged hence output volume will have dimensions  $32 \times 32 \times 12$ .

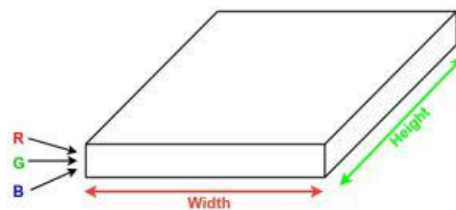
**Pooling layer:** This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are max pooling and average pooling. If we use a max pool with  $2 \times 2$  filters and stride 2, the resultant volume will be of dimension  $16 \times 16 \times 12$ .

**Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.

**Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.

### How Convolutional Layers Works?

Convolution Neural Networks or convnets are neural networks that share their parameters. Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (i.e the channel as images generally have red, green, and blue channels).



Now imagine taking a small patch of this image and running a small neural network, called a filter or kernel on it, with say,  $K$  outputs and representing them vertically. Now slide that neural network across the whole image, as a result, we will get another image with different widths, heights, and depths. Instead of just R, G, and B channels now we have more channels but lesser width and height. **This operation is called Convolution.** If the patch size is the same as that of the image it will be a regular neural network. Because of this small patch, we have fewer weights.

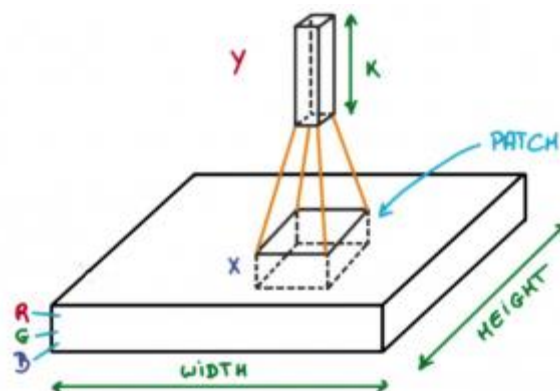




Image size =  $1920 \times 1080 \times 3$

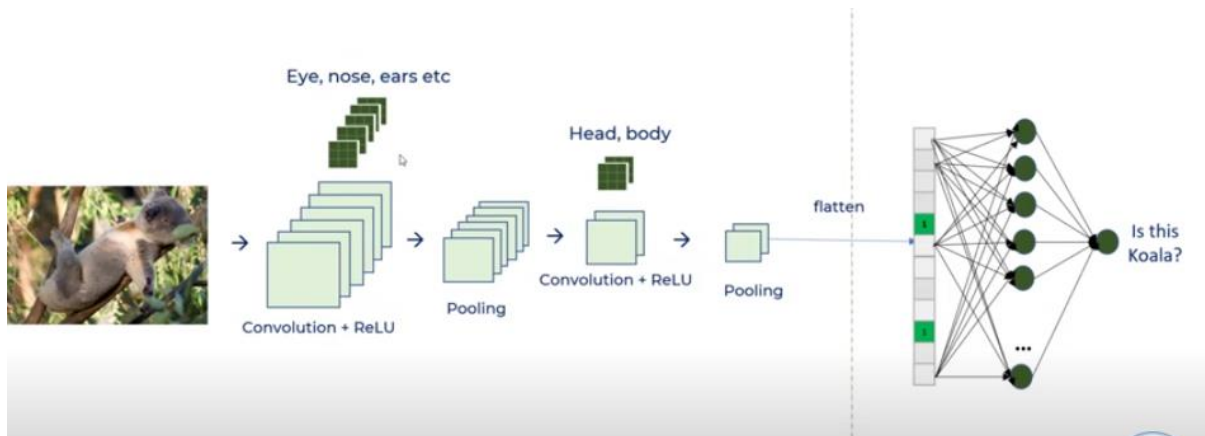
First layer neurons =  $1920 \times 1080 \times 3 \sim 6 \text{ million}$

Hidden layer neurons = Let's say you keep it  $\sim 4 \text{ million}$

Weights between input and hidden layer =  $6 \text{ mil} \times 4 \text{ mil}$   
 $= 24 \text{ million}$



|    |    |    |    |    |
|----|----|----|----|----|
| -1 | 1  | 1  | 1  | -1 |
| -1 | 1  | -1 | 1  | -1 |
| -1 | 1  | 1  | 1  | -1 |
| -1 | -1 | -1 | 1  | -1 |
| -1 | -1 | -1 | 1  | -1 |
| -1 | -1 | 1  | -1 | -1 |
| -1 | 1  | -1 | -1 | -1 |



## Applications of CNNs

- **Image recognition:** CNNs are used to identify objects in images.
- **Object detection:** CNNs are used to detect objects in images and videos.
- **Facial recognition:** CNNs are used to identify faces in images.
- **Medical image analysis:** CNNs are used to analyze medical images, such as those from cancer, ophthalmic diseases, and lung diseases.
- **Autonomous vehicles:** CNNs are used in self-driving cars.

- **Natural language processing:** CNNs can be used for natural language processing tasks.
- **Recommendation engines:** CNNs can be used in recommendation engines.

### Example: Basic CNN for Image Classification

We will assume a grayscale image of size **(28 × 28)** as input and pass it through a CNN with the following layers:

1. **Input Layer:**  $(28 \times 28 \times 1)$
2. **Conv Layer 1:** 6 filters of size  $(3 \times 3)$ , stride = 1, padding = 0
3. **Pooling Layer 1:** Max Pooling  $(2 \times 2)$ , stride = 2
4. **Conv Layer 2:** 16 filters of size  $(3 \times 3)$ , stride = 1, padding = 0
5. **Pooling Layer 2:** Max Pooling  $(2 \times 2)$ , stride = 2
6. **Fully Connected Layer:** 120 neurons
7. **Output Layer:** 10 neurons (for classification)

### Step-by-Step Numerical Example

#### Step 1: Input Layer $(28 \times 28 \times 1)$

The input is a **grayscale** image of shape  $(28 \times 28 \times 1)$ .

#### Step 2: First Convolutional Layer

- **Filter size:**  $(3 \times 3)$
- **Number of filters:** 6
- **Stride:** 1
- **Padding:** 0

Using the formula for output size:

$$\text{Output size} = \frac{(W - F + 2P)}{S} + 1$$

where:

- $W = 28$  (input width)
- $F = 3$  (filter size)
- $P = 0$  (padding)
- $S = 1$  (stride)

$$\text{Output size} = \frac{(28 - 3 + 0)}{1} + 1 = 26 \times 26$$

Since we have **6 filters**, the output volume is:

$$26 \times 26 \times 6$$

**Step 3: First Max Pooling Layer (2 × 2, stride 2)**

$$\text{Output size} = \frac{(26 - 2)}{2} + 1 = 13 \times 13$$

Since depth remains the same (6), the output volume is:

$$13 \times 13 \times 6$$

**Step 4: Second Convolutional Layer**

- Filter size: (3 × 3)
- Number of filters: 16
- Stride: 1
- Padding: 0

$$\text{Output size} = \frac{(13 - 3 + 0)}{1} + 1 = 11 \times 11$$

Since we have **16 filters**, the output volume is:

$$11 \times 11 \times 16$$

**Step 5: Second Max Pooling Layer (2 × 2, stride 2)**

$$\text{Output size} = \frac{(11 - 2)}{2} + 1 = 5 \times 5$$

Since depth remains the same (16), the output volume is:

$$5 \times 5 \times 16$$

**Step 6: Fully Connected Layer**

The feature map is flattened into a **vector**:

$$5 \times 5 \times 16 = 400$$

This vector is fully connected to **120 neurons**, meaning the weight matrix has size (400 × 120).

---

**Step 7: Output Layer**

The fully connected layer is connected to an **output layer** of 10 neurons (for 10 classes), so the final output shape is (10, 1).

### Summary of Shapes at Each Layer

| Layer   | Output Shape |
|---------|--------------|
| Input   | (28, 28, 1)  |
| Conv 1  | (26, 26, 6)  |
| Pool 1  | (13, 13, 6)  |
| Conv 2  | (11, 11, 16) |
| Pool 2  | (5, 5, 16)   |
| Flatten | (400)        |
| FC 1    | (120)        |
| Output  | (10)         |

## VGG-Net Architecture

The Visual Geometry Group (VGG) models, particularly **VGG-16** and **VGG-19**, have significantly influenced the field of computer vision since their inception. These models, introduced by the **Visual Geometry Group** from the **University of Oxford**, stood out in the **2014 ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** for their deep convolutional neural networks (CNNs) with a uniform architecture. VGG-19, the deeper variant of the VGG models, has garnered considerable attention due to its simplicity and effectiveness.

### Evolution of VGG Models

Before the advent of VGG models, CNN architectures like **LeNet-5** and **AlexNet** laid the groundwork for deep learning in computer vision. LeNet-5, introduced in the 1990s, was one of the first successful applications of CNNs in recognizing handwritten digits. AlexNet, which won the ILSVRC in 2012, marked a significant breakthrough by leveraging deeper architectures and GPU acceleration.

The VGG models were introduced by Karen Simonyan and Andrew Zisserman in their 2014 paper titled "Very Deep Convolutional Networks for Large-Scale Image Recognition." The primary objective was **to investigate the effect of increasing the depth of CNNs on large-scale image recognition tasks**. VGG-16 and VGG-19, with **16 and 19 weight layers** respectively, were among the most notable models presented in the paper. Their design was characterized by using small 3x3 convolution filters consistently across all layers, which simplified the network structure and improved performance.

### VGG-19 Architecture

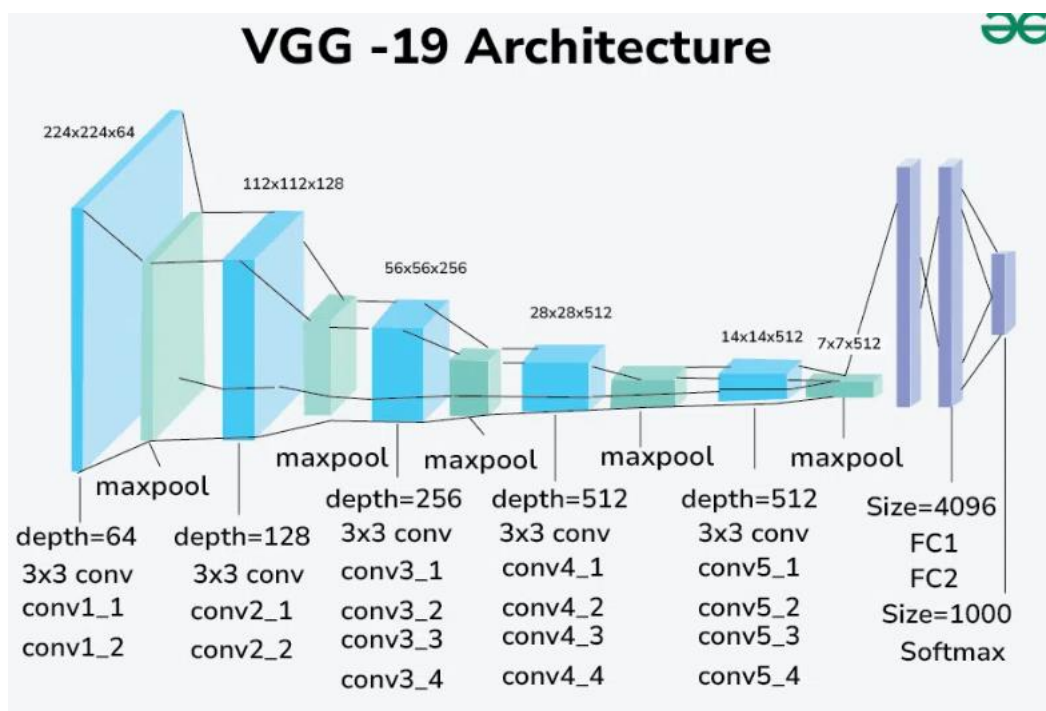
VGG-19 is a deep convolutional neural network with **19 weight layers**, comprising **16 convolutional layers and 3 fully connected layers**. The architecture follows a straightforward and repetitive pattern, making it easier to understand and implement.



**(a)** Architecture of VGG19 model

The key components of the VGG-19 architecture are:

- Convolutional Layers: 3x3 filters with a stride of 1 and padding of 1 to preserve spatial resolution.
- Activation Function: ReLU (Rectified Linear Unit) applied after each convolutional layer to introduce non-linearity.
- Pooling Layers: Max pooling with a 2x2 filter and a stride of 2 to reduce the spatial dimensions.
- Fully Connected Layers: Three fully connected layers at the end of the network for classification.
- Softmax Layer: Final layer for outputting class probabilities.



## Features of VGGNet-19

- **Model Simplicity and Effectiveness:** The VGG-19 architecture's simplicity, characterized by its uniform use of 3x3 convolution filters and repetitive block structure, makes it a highly effective and easy-to-implement model for various computer vision tasks.
- **Computational Requirements:** One of the key trade-offs of the VGG-19 model is its computational demand. Due to its depth and the use of small filters, it requires



significant memory and computational power, making it more suited for environments with robust hardware capabilities.

- **Robust Feature Extraction:** The depth of the VGG-19 model allows it to capture intricate features in images, making it an excellent feature extractor. This capability is particularly useful in transfer learning, where pre-trained VGG-19 models are fine-tuned for specific tasks, leveraging the rich feature representations learned from large datasets.
- **Data Augmentation:** To enhance the performance and generalization capability of VGG-19, data augmentation techniques such as random cropping, horizontal flipping, and color jittering are often employed during training. These techniques help the model to better handle variations and improve its robustness.
- **Influence on Network Design:** The principles established by the VGG-19 architecture, such as the use of small convolution filters and deep networks, have influenced the design of subsequent state-of-the-art models. Researchers have built upon these concepts to develop more advanced architectures that continue to push the boundaries of what is possible in computer vision.

## Stride and Padding

In a convolutional neural network (CNN), padding and stride are parameters that affect how the convolution operation is performed.

Padding: - padding is a technique used **to preserve the spatial dimensions** of the input image **after convolution operations on a feature map**. Padding involves adding extra pixels around the border of the input feature map before convolution.

- Adds extra pixels, usually zeros, around the image's border
- Increases the processing region of the CNN
- Helps the kernel process the image by providing more room to cover it
- Preserves spatial information, such as edges and corners
- Allows deeper networks by maintaining consistent dimensions across layers

Stride :- stride is the number of pixels by which a filter moves across an input image. It's a key parameter in the convolution operation, which is a fundamental building block of CNNs.

- The number of pixels by which the filter moves across the input image
- Determines how the filter convolves over the input matrix
- Can be used to down-sample the output image
- Can reduce the number of parameters
- Can increase the computational efficiency of the convolutional layer

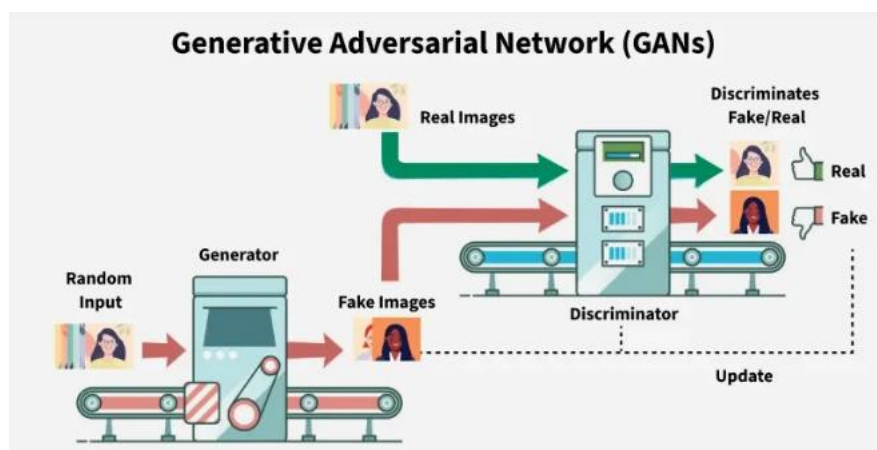
# Generative Adversarial Network (GAN)

Generative Adversarial Networks (GANs) were introduced by Ian Goodfellow and his colleagues in 2014. GANs are a class of neural networks that autonomously learn patterns in the input data to generate new examples resembling the original dataset.

GAN's architecture **consists of two neural networks**:

- **Generator**: creates synthetic data from random noise to produce data so realistic that the discriminator cannot distinguish it from real data.
- **Discriminator**: acts as a critic, evaluating whether the data it receives is real or fake.

They use adversarial training to produce artificial data that is identical to actual data.



## Detailed Architecture of GANs

### 1. Generator Model

The generator is a deep neural network that takes random noise as input to generate realistic data samples (e.g., images or text). It learns the underlying data distribution by adjusting its parameters through backpropagation.

The generator's objective is to produce samples that the discriminator classifies as real. The loss function is:

$$J_G = -\frac{1}{m} \sum_{i=1}^m \log D(G(z_i))$$

Where,

- $J_G$  measure how well the generator is fooling the discriminator.
- $\log D(G(z_i))$  represents log probability of the discriminator being correct for generated samples.
- The generator aims to minimize this loss, encouraging the production of samples that the discriminator classifies as real ( $\log D(G(z_i))$ , close to 1).

### 2. Discriminator Model

The discriminator acts as a binary classifier, distinguishing between real and generated data. It learns to improve its classification ability through training, refining its parameters to detect fake samples more accurately.

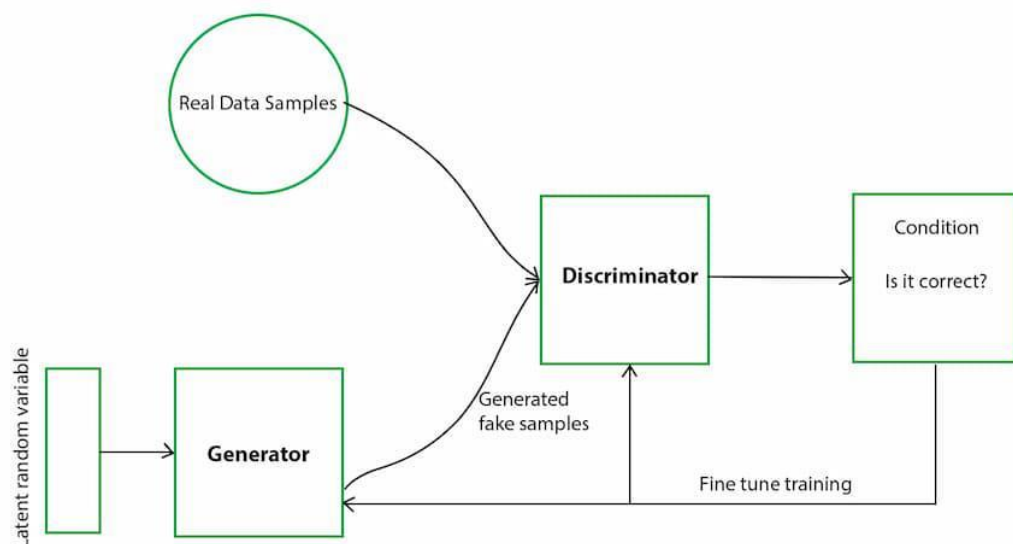
When dealing with image data, the discriminator often employs convolutional layers or other relevant architectures suited to the data type. These layers help extract features and enhance the model's ability to differentiate between real and generated samples.

The discriminator reduces the negative log likelihood of correctly classifying both produced and real samples. This loss incentivizes the discriminator to accurately categorize generated samples as fake and real samples with the following equation:

$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$

- $J_D$  assesses the discriminator's ability to discern between produced and actual samples.
- The log likelihood that the discriminator will accurately categorize real data is represented by  $\log D(x_i)$ .
- The log chance that the discriminator would correctly categorize generated samples as fake is represented by  $\log(1 - D(G(z_i)))$ .

The generator aims to minimize the loss, while the discriminator tries to maximize its classification accuracy.



## How does a GAN work?

Let's understand how the generator (G) and discriminator (D) compete to improve each other over time:

### 1. Generator's First Move

G takes a random noise vector as input. This noise vector contains random values and acts as the starting point for G's creation process. Using its internal layers and learned patterns, G transforms the noise vector into a new data sample, like a generated image.

### 2. Discriminator's Turn

D receives two kinds of inputs:

Real data samples from the training dataset.

The data samples generated by G in the previous step.

D's job is to analyze each input and determine whether it's real data or something G cooked up. It outputs a probability score between 0 and 1. A score of 1 indicates the data is likely real, and 0 suggests it's fake.

### **3. Adversarial Learning**

If the discriminator correctly classifies real data as real and fake data as fake, it strengthens its ability slightly.

If the generator successfully fools the discriminator, it receives a positive update, while the discriminator is penalized.

### **4. Generator's Improvement**

Every time the discriminator misclassifies fake data as real, the generator learns and improves. Over multiple iterations, the generator produces more convincing synthetic samples.

### **5. Discriminator's Adaptation**

The discriminator continuously refines its ability to distinguish real from fake data. This ongoing duel between the generator and discriminator enhances the overall model's learning process.

### **6. Training Progression**

As training continues, the generator becomes highly proficient at producing realistic data.

Eventually, the discriminator struggles to distinguish real from fake, indicating that the GAN has reached a well-trained state.

At this point, the generator can be used to generate high-quality synthetic data for various applications.

## **Types of GANs**

### **1. Vanilla GAN**

Vanilla GAN is the simplest type of GAN. It consists of: A generator and a discriminator, both are built using multi-layer perceptrons (MLPs).

The model optimizes its mathematical formulation using stochastic gradient descent (SGD).

While Vanilla GANs serve as the foundation for more advanced GAN models, they often struggle with issues like mode collapse and unstable training.

### **2. Conditional GAN (CGAN)**

Conditional GANs (CGANs) introduce an additional conditional parameter to guide the generation process. Instead of generating data randomly, CGANs allow the model to produce specific types of outputs.

Working of CGANs:

A conditional variable ( $y$ ) is fed into both the generator and the discriminator.

This ensures that the generator creates data corresponding to the given condition (e.g., generating images of specific objects).

The discriminator also receives the labels to help distinguish between real and fake data.

### **3. Deep Convolutional GAN (DCGAN)**

Deep Convolutional GANs (DCGANs) are among the most popular and widely used types of GANs, particularly for image generation.

What Makes DCGAN Special?

Uses Convolutional Neural Networks (CNNs) instead of simple multi-layer perceptrons (MLPs).

Max pooling layers are replaced with convolutional stride, making the model more efficient.

Fully connected layers are removed, allowing for better spatial understanding of images.

DCGANs have been highly successful in generating high-quality images, making them a go-to choice for deep learning researchers.

### **4. Laplacian Pyramid GAN (LAPGAN)**

Laplacian Pyramid GAN (LAPGAN) is designed to generate ultra-high-quality images by leveraging a multi-resolution approach.

Working of LAPGAN:

Uses multiple generator-discriminator pairs at different levels of the Laplacian pyramid.

Images are first downsampled at each layer of the pyramid and upsampled again using Conditional GANs (CGANs).

This process allows the image to gradually refine details, reducing noise and improving clarity.

Due to its ability to generate highly detailed images, LAPGAN is considered a superior approach for photorealistic image generation.

### **5. Super Resolution GAN (SRGAN)**

Super-Resolution GAN (SRGAN) is specifically designed to increase the resolution of low-quality images while preserving details.

Working of SRGAN:

Uses a deep neural network combined with an adversarial loss function.

Enhances low-resolution images by adding finer details, making them appear sharper and more realistic.

Helps reduce common image upscaling errors, such as blurriness and pixelation.

## What is LSTM – Long Short Term Memory?

Long Short-Term Memory (LSTM) is an enhanced version of the Recurrent Neural Network (RNN) designed by Hochreiter & Schmidhuber. LSTMs can capture long-term dependencies in sequential data making them ideal for tasks like language translation, speech recognition and time series forecasting.

### LSTM Architecture

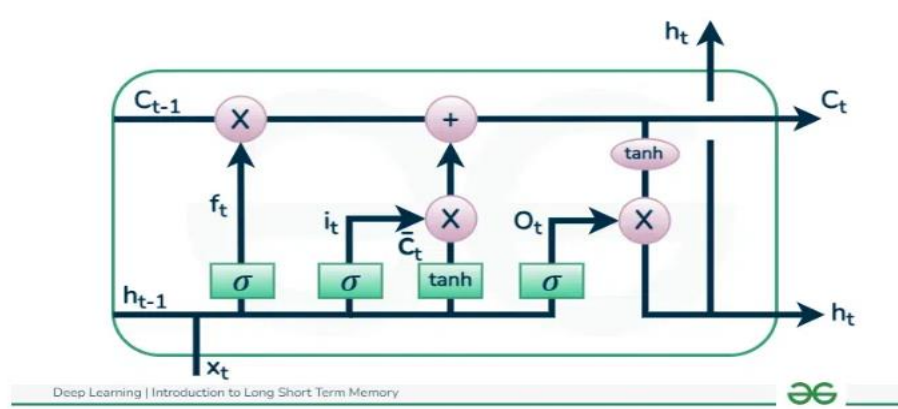
LSTM architectures involves the memory cell which is controlled by three gates: the input gate, the forget gate and the output gate. These gates decide what information to add to, remove from and output from the memory cell.

- Input gate: Controls what information is added to the memory cell.
- Forget gate: Determines what information is removed from the memory cell.
- Output gate: Controls what information is output from the memory cell.

This allows LSTM networks to selectively retain or discard information as it flows through the network which allows them to learn long-term dependencies. The network has a hidden state which is like its short-term memory. This memory is updated using the current input, the previous hidden state and the current state of the memory cell.

### Working of LSTM

LSTM architecture has a chain structure that contains four neural networks and different memory blocks called cells.



### Applications of LSTM

Some of the famous applications of LSTM includes:

- **Language Modeling:** Used in tasks like language modeling, machine translation and text summarization. These networks learn the dependencies between words in a sentence to generate coherent and grammatically correct sentences.
- **Speech Recognition:** Used in transcribing speech to text and recognizing spoken commands. By learning speech patterns they can match spoken words to corresponding text.
- **Time Series Forecasting:** Used for predicting stock prices, weather and energy consumption. They learn patterns in time series data to predict future events.

- **Anomaly Detection:** Used for detecting fraud or network intrusions. These networks can identify patterns in data that deviate drastically and flag them as potential anomalies.
- **Recommender Systems:** In recommendation tasks like suggesting movies, music and books. They learn user behaviour patterns to provide personalized suggestions.
- **Video Analysis:** Applied in tasks such as object detection, activity recognition and action classification. When combined with Convolutional Neural Networks (CNNs) they help analyse video data and extract useful information.

## LSTM vs RNN

| Feature                          | LSTM (Long Short-term Memory)                                                                                     | RNN (Recurrent Neural Network)                                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Memory                           | Has a special memory unit that allows it to learn long-term dependencies in sequential data                       | Does not have a memory unit                                                                              |
| Directionality                   | Can be trained to process sequential data in both forward and backward directions                                 | Can only be trained to process sequential data in one direction                                          |
| Training                         | More difficult to train than RNN due to the complexity of the gates and memory unit                               | Easier to train than LSTM                                                                                |
| Long-term dependency learning    | Yes                                                                                                               | Limited                                                                                                  |
| Ability to learn sequential data | Yes                                                                                                               | Yes                                                                                                      |
| Applications                     | Machine translation, speech recognition, text summarization, natural language processing, time series forecasting | Natural language processing, machine translation, speech recognition, image processing, video processing |